Session 11 : lab

Create a Zoo Animal Tracker in Kotlin that uses inheritance, abstract classes, and interfaces. Simulate various animal actions like roaring, spraying water, and mimicking words. This task strengthens your understanding of polymorphism and reusable class structures.

```
//  Zoo Animal Tracker


// Abstract Class
abstract class Animal(val name: String) {
    abstract fun makeSound()
}


//  Interface
interface Feedable {
    fun feed(food: String)
}


//  Lion
class Lion(name: String) : Animal(name), Feedable {


    override fun makeSound() {
        println("$name: Roar!")
    }
```

```
    fun hunt() {
        println("$name is hunting in the savannah.")
    }

    override fun feed(food: String) {
        println("$name eats $food.")
    }
}

// Subclass: Elephant
class Elephant(name: String) : Animal(name) {

    override fun makeSound() {
        println("$name: Trumpet!")
    }

    fun sprayWater() {
        println("$name sprays water from its trunk.")
    }
}

// 2️⃣ Subclass: Parrot
class Parrot(name: String) : Animal(name), Feedable {
```

```
    override fun makeSound() {
        println("$name: Chirp!")
    }

    fun fly() {
        println("$name is flying around the zoo.")
    }

    override fun feed(food: String) {
        println("$name pecks at $food.")
    }
}

// 4️⃣ Function to Display Zoo Animals
fun displayZoo(animals: List<Animal>) {
    println("--- Zoo Tracker ---")
    for (animal in animals) {
        animal.makeSound()  // Polymorphism in action
    }
}

// 5️⃣ Main Function
fun main() {

    val lion = Lion("Lion")
```

```
    val elephant = Elephant("Elephant")
    val parrot = Parrot("Parrot")

    val zooAnimals: List<Animal> = listOf(lion, elephant, parrot)

    displayZoo(zooAnimals)

    println()

    // Feeding some animals
    lion.feed("meat")
    parrot.feed("seeds")

    println()

    // Calling extra behaviors
    elephant.sprayWater()
    lion.hunt()
    parrot.fly()
}
```